THE STUDENT AI PROMPT TOOLKIT

A practical field guide to working with AI while you build.

09

SPECIALIZED PROMPTS / ONE PROMPT PER PAGE

DEBUG / CREATE / IMPROVE / TEST
For beginners, student developers, hackathons, and vibe-coding workshops.

How to use this guide

A good prompt gives an AI assistant enough context to do useful work — without leaving important decisions to guesswork.

01 Replace every [bracketed field] with details from your own project.

02 Share relevant code, screenshots, logs, or examples when they help.

03 Give the AI one clear task at a time.

04 Ask it to inspect before editing, especially in an existing project.

05 Review the changes and run the app or tests yourself.

A prompt should answer five questions

- What are you building? — Project context and tech stack.
- What do you need? — One specific task or outcome.
- What should it look like? — Requirements and expected behavior.
- What must stay safe? — Constraints, existing features, and data.
- How will you know it worked? — Verification and success criteria.

A quick safety check

Never paste passwords, API keys, private student information, or other secrets into an AI tool. Do not assume code works because it looks correct. Run it, test it, and understand what you submit.

01 / DEBUGGING

Find the cause.
Fix it carefully.

USE THIS WHEN When something crashes, returns an error, or behaves differently than expected.

COPY-READY PROMPT

Act as a careful senior software engineer and debugging specialist.
PROJECT
[Project description and tech stack]
PROBLEM
[Describe what is going wrong]
ERROR MESSAGE / LOGS
[Paste the complete error message and relevant stack trace]
EXPECTED BEHAVIOR
[What should happen?]
ACTUAL BEHAVIOR
[What happens instead?]
DEBUGGING APPROACH
1. Inspect the relevant code before editing.
2. Trace the issue to its root cause. Do not guess.
3. Explain the cause in plain language.
4. Propose the smallest reliable fix.
5. Make only the changes needed to solve the issue.
6. Check nearby code for possible side effects.
CONSTRAINTS
- Do not hide or suppress the error to make it disappear.
- Do not rewrite working code unnecessarily.
- Do not invent files, functions, or test results.
- Ask me if critical information is missing.
VERIFY
Run relevant checks if available. Report what you actually tested, what passed, what failed, and what I should verify manually.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

02 / WEBSITE CREATION

Start with the idea. Build with intention.

USE THIS WHEN When creating a portfolio, landing page, website, or web application from scratch.

COPY-READY PROMPT

Act as a senior full-stack developer and thoughtful UI/UX designer.
PROJECT
Name: [Website or app name]
Purpose: [What should it help people do?]
Target audience: [Who will use it?]
Tech stack: [Preferred tools, or recommend a suitable stack]
STRUCTURE & CONTENT
Pages: [List required pages]
Main content: [What belongs on each page?]
Core features: [List important functionality]
DESIGN DIRECTION
Visual style: [Minimal / playful / editorial / futuristic / etc.]
Colors and typography: [Preferences]
References: [Links or descriptions of examples]
BUILD REQUIREMENTS
- Create a consistent design system and reusable components.
- Make the layout responsive on mobile, tablet, and desktop.
- Use semantic, accessible markup and clear navigation.
- Make buttons, forms, links, and controls functional.
- Include loading, empty, and error states where relevant.
- Use maintainable code and appropriate types.
- Avoid unnecessary dependencies and fake functionality.
WORKFLOW
First inspect the project and propose the structure, design direction, and implementation plan. Then build in logical stages. Ask before making decisions that would materially change the result.
At the end, summarize what works, what remains incomplete, and how to run the project.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

03 / FEATURE BUILDING

Add one capability.
Keep the rest working.

USE THIS WHEN When adding login, search, filters, profiles, dashboards, or another feature to an existing project.

COPY-READY PROMPT

Act as a senior full-stack developer working inside an existing project.
EXISTING PROJECT
[Describe the app, architecture, and tech stack]
FEATURE
[Feature name and purpose]
USER STORY
As a [type of user], I want to [action] so that [benefit].
EXPECTED BEHAVIOR
1. [What the user does]
2. [What the system should do]
3. [What the user sees or can do next]
DATA & RULES
Data required: [Fields or sources]
Validation / permissions: [Rules]
Edge cases: [Empty values, duplicates, failures, etc.]
IMPLEMENTATION RULES
- Inspect the existing project before editing.
- Reuse established patterns and components where appropriate.
- Keep the feature consistent with the current design.
- Handle loading, success, and error states.
- Preserve existing features and integrations.
- Do not modify unrelated files or public interfaces without explaining why.
First identify the files and systems likely to be affected. Then implement in small, reviewable steps and verify the main user flow.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

04 / UI / UX IMPROVEMENT

Make it clearer.
Make it easier to use.

USE THIS WHEN When a screen feels cluttered, inconsistent, hard to navigate, or visually unfinished.

COPY-READY PROMPT

Act as an experienced product designer and frontend engineer.
INTERFACE
[Page, screen, or component]
Target users: [Who uses it?]
Current problems: [What feels confusing, cluttered, or unfinished?]
DESIRED EXPERIENCE
Style / mood: [Describe the intended look]
References: [Optional websites or screenshots]
Must preserve: [Brand elements, content, and functionality]
REVIEW
- Information hierarchy and page structure
- Spacing, alignment, and visual consistency
- Typography, contrast, and color use
- Navigation and interaction clarity
- Mobile and tablet responsiveness
- Accessibility and readable labels
- Feedback for hover, focus, loading, success, and errors
INSTRUCTIONS
1. Review the current interface before changing it.
2. Identify the highest-impact usability and visual issues.
3. Propose focused improvements.
4. Implement without breaking existing functionality.
5. Avoid effects that make the interface slower or harder to use.
Do not redesign unrelated screens. Summarize the improvements and any remaining accessibility or responsive issues.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

05 / REFACTORING

Improve the code.
Preserve its behavior.

USE THIS WHEN When code is repetitive, difficult to understand, slow, or hard to maintain.

COPY-READY PROMPT

Act as a senior software engineer focused on maintainability and performance.
PROJECT
[Project description and tech stack]
Files / modules: [Identify the scope]
Main concern: [Readability, duplication, performance, structure, etc.]
GOALS
- Reduce unnecessary complexity and duplication.
- Improve readability and separation of concerns.
- Improve performance where there is a measurable reason.
- Follow the project's existing conventions.
- Make the code easier to test and extend.

CONSTRAINTS
- Preserve current behavior and expected outputs.
- Do not change public APIs unless necessary and approved.
- Avoid adding dependencies without a clear benefit.
- Do not refactor unrelated areas.
- Keep changes small enough to review.

PROCESS
1. Inspect the relevant code.
2. Identify specific problems and their impact.
3. Recommend a focused refactoring plan.
4. Implement the approved changes.
5. Run relevant tests or checks and compare behavior.
Report files changed, improvements, trade-offs, and anything that could not be verified. Do not claim performance gains without measurements.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

06 / BACKEND & DATABASE

Design the system. Protect the data.

USE THIS WHEN When building APIs, database models, authentication, or server-side logic.

COPY-READY PROMPT

Act as a senior backend engineer focused on secure, maintainable systems.
APPLICATION
[What does the application do?]
Backend stack: [Framework and language]
Database: [Database and relevant tools]
Existing architecture: [Routes, services, models, or patterns]
TASK
[Describe the API, database, or backend functionality]
DATA & BUSINESS RULES
Entities and fields: [Describe required data]
Relationships: [How records connect]
Validation: [Required fields, formats, limits]
Permissions: [Who can perform each action?]
Failure cases: [Invalid input, missing record, outage, etc.]
REQUIREMENTS
- Inspect the existing schema and conventions first.
- Validate input on the server.
- Enforce authentication and authorization where needed.
- Use safe database operations and protect secrets.
- Handle errors consistently.
- Consider data integrity, indexes, and performance.
- Do not assume destructive migrations are safe.

Explain the design and migration impact first. Then implement the smallest suitable solution and test successful, invalid, and unauthorized requests where applicable.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

07 / CODE REVIEW

Catch issues before they ship.

USE THIS WHEN Before a pull request, project submission, demo, or release.

COPY-READY PROMPT

Act as a senior engineer reviewing code before it is merged or released.
CONTEXT
Project / tech stack: [Details]
Purpose of this change: [What should the code accomplish?]
Files or change set: [Identify the scope]
Known concerns: [Optional]
REVIEW FOR
1. Functional bugs and incorrect assumptions.
2. Security and privacy risks.
3. Performance problems.
4. Error handling and input validation.
5. Maintainability and architecture.
6. Accessibility or usability issues, where relevant.
7. Missing tests and important edge cases.
FOR EACH FINDING, PROVIDE
- Severity: Critical / High / Medium / Low.
- File and exact location, if available.
- What could go wrong.
- Why it matters.
- A concrete recommended fix.

RULES
- Prioritize real defects over personal style preferences.
- Do not invent issues or file locations.
- Separate confirmed problems from possible risks.
- Do not modify code during this review.

Finish with an overall assessment and prioritized fixes. Wait for my approval before implementing changes.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

08 / TESTING & EDGE CASES

Test the happy path.
Then test what can go wrong.

USE THIS WHEN When you need confidence in a function, component, API, or user workflow.

COPY-READY PROMPT

Act as a quality-focused test engineer.
SYSTEM UNDER TEST
[Function, component, API, or user workflow]
Expected behavior: [Describe correct outcomes]
Test framework: [Jest, Vitest, xUnit, etc.]
Existing test conventions: [Describe or identify them]
CREATE TESTS FOR
- Normal successful behavior.
- Boundary and limit values.
- Missing, empty, null, or malformed input.
- Invalid user actions or state transitions.
- Permission and authentication failures, if applicable.
- Network, API, or database failures, if applicable.
- Duplicate or repeated actions.
- Important regression scenarios.

FOR EACH TEST, DEFINE
1. Scenario and purpose.
2. Setup and input.
3. Expected result.
IMPLEMENTATION RULES
- Follow the existing test structure.
- Keep tests independent and repeatable.
- Avoid tests that only duplicate implementation details.
- Do not change production behavior just to make a test pass.

Run tests if execution is available. Report the exact command, results, failures, and tests not run. Never claim a test passed unless it actually ran successfully.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

09 / UNDERSTAND A CODEBASE

Learn how it works before changing it.

USE THIS WHEN When joining a team project, exploring a new repository, or learning an unfamiliar codebase.

COPY-READY PROMPT

Act as a patient senior developer and programming mentor.
PROJECT
[Project name or description]
Known tech stack: [List technologies, or identify them from the files]
My experience level: [Beginner / intermediate / advanced]
Learning goal: [What do I want to understand?]
TEACH ME THE CODEBASE
1. Explain what the application does and how it starts.
2. Map the important folders and files.
3. Explain the architecture and major responsibilities.
4. Trace one important user flow from input to output.
5. Explain how components, services, APIs, and database models connect.
6. Identify important dependencies and configuration.
7. Point out where a beginner should start reading.
TEACHING STYLE
- Use plain language and define unfamiliar terms.
- Explain why the code is structured this way.
- Refer to actual file names and symbols.
- Use small examples grounded in the project.
- Separate confirmed observations from assumptions.

Do not modify files. Finish with an architecture summary, a glossary of important terms, and three questions to check my understanding.

REMEMBER Treat AI output as a draft. Review it, run it, and make sure you can explain it.

Build with curiosity.

Review with care. Learn by doing.

Before you call it done

- ■ I understand what the generated code is intended to do.
- ■ I reviewed the files and changes before accepting them.
- ■ I ran the app and tested the important user flows.
- ■ I checked errors and edge cases.
- ■ I protected passwords, API keys, and private information.
- ■ I can explain what I built and how AI helped.

Try this follow-up

“Explain the solution you just implemented. Walk me through the key decisions, files changed, possible limitations, and how I can test it myself. Assume I am learning, and do not skip important concepts.”